Link Prediction for Microservice Call Graphs: Temporal Windows and Scalability Trade-offs

Anonymous Author(s)

Abstract

Modern microservice systems generate dynamic interaction patterns that evolve rapidly over time, making it challenging to maintain reliability and scalability. Link prediction, by anticipating future service interactions, supports adaptive monitoring, fault prevention, and efficient resource management. In this paper, we investigate link prediction across model architectures, temporal drift, real-time feasibility, and cross-dataset portability. We conduct extensive experiments across three real-world datasets (Alibaba 2021, Alibaba 2022, and Huawei 2021), comparing baseline models such as Random Walk and LSTM with more advanced approaches including Graph Neural Networks, Diffusion Models, and Transformers. Our results show that the enhanced diffusion model with GAT model achieves the best trade-off between performance and efficiency, reaching an F1-score of 0.923, precision of 0.877, recall of 0.975, and ROC-AUC of 0.970. We further analyze the effect of time window sizes, overlapping intervals, and increasing gaps between training and testing. Our findings assess the capabilities and constraints of existing predictive methods. They also guide the development of scalable, time-aware forecasting systems for microservices.

Keywords

Link Prediction, Microservice, Graph Neural Networks, Graph.

ACM Reference Format:

Anonymous Author(s). 2026. Link Prediction for Microservice Call Graphs: Temporal Windows and Scalability Trade-offs . In *Proceedings of Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE 2026) (ICSE '26).* ACM, New York, NY, USA, 11 pages.

1 Introduction

Microservices architecture has become foundational in modern software systems due to its advantages in scalability, modularity, and ease of deployment. However, as these systems grow in size and complexity, understanding and managing service-to-service interactions becomes critical to maintaining performance and reliability [6].

We propose *link prediction* as a proactive technique to anticipate future interactions within microservice call graphs. This involves forecasting which services are likely to communicate in the near future based on past patterns and system dynamics. As illustrated in Figure 1, link prediction enables systems to infer new edges in the call graph at time t+1, based on observations at time t, thereby supporting early anomaly detection, resource provisioning, and adaptive monitoring. This temporal prediction capability is particularly important in real-world microservice environments because interactions among services are highly dynamic and time-sensitive, directly influencing system performance, reliability, and resource management.

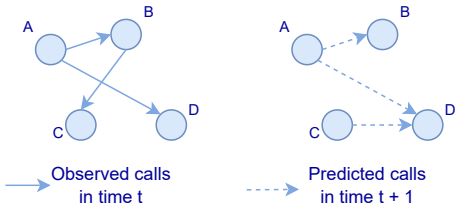

Figure 1: Solid arrows show observed calls at time t; dashed arrows show predicted calls at time t+1.

Despite its potential, link prediction in microservice systems remains underexplored. Prior work often targets static knowledge graphs [29] or performance forecasting [12], while neglecting temporal and structural dynamics crucial to microservices. Others evaluate on constrained or synthetic data, limiting real-world applicability [9, 21, 31].

To address this gap, we conduct an evaluation of temporal link prediction across three production-scale datasets: Alibaba 2021 [19], Alibaba 2022 [14], and Huawei Cloud 2021 [5]. We evaluate a variety of models, from classical baselines like Random Walk and LSTM to advanced architectures such as GNNs, Diffusion models, and Transformers. We also include hybrid models like Diffusion-GAT and Transformer-GAT, which combine structural and temporal learning strengths. Our study focuses on two main objectives: (1) benchmarking model performance in realistic settings, and (2) analyzing how operational factors such as time windows, prediction gaps, and training drift affect predictive reliability.

In addition to model comparisons, we evaluate how temporal segmentation strategies (such as overlapping vs. gapped windows) and test-time conditions impact generalization. We also assess model scalability and real-time feasibility, exploring how inference latency aligns with microservice monitoring requirements.

Our key contributions are as follows:

- We propose a temporal link prediction framework tailored to microservice call graphs, including negative sampling, windowing, and timestamp-aware GAT construction.
- We benchmark diverse predictive models on three real-world datasets, analyzing accuracy, efficiency trade-offs and robustness to dataset variability.

ICSE '26, Rio de Janeiro, Brazil

- We evaluate model performance across time-window sizes, temporal gaps, and train-test drift, showing that Diffusion+GAT generalizes better than Transformers with lower computational cost.
- We assess real-time feasibility and expose latency bottlenecks that limit deployment in fast-evolving microservice environments.

The paper proceeds as follows: Section 2 reviews related work; Section 3 describes our problem setup and models; Section 4 presents empirical results; Section 5 analyzes key findings; Section 6 discusses limitations; and Section 7 concludes with future directions.

2 Background and Related Work

Microservices architecture decomposes monolithic applications into smaller, independently deployable services, significantly improving scalability, flexibility, and maintainability [6, 27]. In such systems, services frequently communicate with one another through lightweight mechanisms such as RESTful APIs or message queues. These communications, known as interactions, may involve request-response calls, data sharing, or dependency invocations that occur during the execution of distributed business logic [10].

A call graph structurally represents service interactions, where nodes are microservices and directed edges indicate runtime calls between them [39]. As systems evolve, these graphs become highly dynamic due to deployment changes, load balancing, autoscaling, and fault recovery [25]. Analyzing such graphs is essential for identifying dependencies, monitoring behavior, and predicting future interactions, key tasks for ensuring reliability and enabling proactive management [4, 40].

However, reliably predicting future interactions in microservice call graphs is particularly challenging due to their temporal volatility and large scale. Accurate link prediction under these conditions is essential to ensure timely adaptation, minimize downtime, and maintain service-level objectives [7, 22].

Link prediction, a core problem in network science, aims to estimate the likelihood of future or missing edges in a graph based on its historical structure [23, 24]. It has demonstrated strong performance in domains such as social networks [38], biological systems [26], and recommendation engines [37], where structural evolution reflects meaningful patterns. These successes have inspired efforts to apply link prediction techniques to microservice call graphs, where forecasting interactions could improve observability and performance [7, 22].

Despite this potential, many traditional methods fail to meet the demands of microservice environments. Heuristic-based techniques like Common Neighbors [23], Adamic-Adar [1], and Node2Vec [15] perform well in static settings but lack the ability to model the dynamic, time-sensitive behavior of distributed systems [28]. Microservices often exhibit short-lived and bursty interactions along with shifting topologies, which these models struggle to capture effectively.

To incorporate temporal information, sequential models such as Long Short-Term Memory (LSTM) networks were introduced for dynamic graphs due to their capacity to learn time-dependent patterns [17]. Hybrid models like Graph Convolutional LSTM (GC-LSTM) further integrated structural and temporal reasoning [3]. Yet, these methods tend to face performance bottlenecks when applied to large-scale microservice data, where high-frequency interactions and real-time constraints are common.

To address these limitations, Graph Neural Networks (GNNs) emerged as scalable and expressive tools for learning on graph-structured data. In particular, Graph Attention Networks (GATs) [34] dynamically weigh neighboring nodes, improving performance in heterogeneous graphs. Prior work [2] has adapted GATs for microservice prediction by incorporating timestamps into the attention mechanism, yielding significant improvements over static baselines. Additionally, diffusion-based models [21] enhance temporal stability by smoothing signals across the graph, helping mitigate noise and fluctuation in service interactions.

More recently, transformer-based architectures, known for their effectiveness in modeling long-range dependencies, have been extended to graph settings [8]. These models enable global attention across time and topology, overcoming the locality constraints of GNNs.

Complementary work in software performance modeling further highlights the utility of graph-based models. For example, PERT-GNN [31] uses GNNs to predict latency in distributed systems, reinforcing the broader applicability of temporal graph learning in software engineering contexts.

In summary, existing graph-based models often assume static graphs, ignore short-term temporal shifts, or fail to scale for high-throughput microservices. This work addresses these gaps by proposing a tailored architecture and evaluating it across varied temporal and operational scenarios.

3 Methodology

Our research investigates the feasibility and effectiveness of predicting future microservice interactions by modeling link formation as a temporal graph learning problem. The core problem we address is as follows: *Given a sequence of evolving call graphs over time (reflecting dynamic service interactions), can we accurately predict future interactions between microservices?* We frame this link prediction task as binary classification over pairs of nodes within time-sliced graphs.

To structure our study, we formulated six research questions that address core challenges in temporal link prediction for microservice call graphs: **RQ1:** Which model performs best for link prediction in microservice call graphs? **RQ2:** How do different time windowing strategies impact model performance on dynamic call graphs? **RQ3:** Are there recurring patterns in microservice connections over time? **RQ4:** How does training on one time period and testing on another affect model generalization, especially under increasing temporal gaps? **RQ5:** How do different datasets from various systems influence model performance? **RQ6:** Can we predict links in real-time, and which model is most suitable for live predictions?

These questions guide the design of our methodology and evaluation.

3.1 Datasets and Graph Construction

We use three real-world datasets from large-scale microservice platforms. The primary dataset is Alibaba 2022 [20], selected for its

clean and detailed traces. To evaluate generalization, we include Alibaba 2021 [19], with similar structure but more temporal variability, and Huawei Cloud 2021 [36], which is noisier and less structured.

All datasets are processed using a unified pipeline that filters invalid entries and encodes each call into a standardized CSV format with timestamp, source (`um`), and destination (`dm`) microservice fields. We apply a **temporal windowing strategy** to segment each dataset into time slices. Within each window, we construct a directed graph where nodes are microservices and edges represent calls. Timestamps are retained as edge attributes.

Each graph is represented as a PyTorch Geometric object $G_t = (x, edge\_index, edge\_attr)$, where: `x` is a learnable microservice embedding, `edge_index` encodes directed links, and `edge_attr` stores edge timestamps. Multiple calls between the same pair are preserved to maintain fine-grained interaction fidelity.

Timestamps in `edge_attr` enable time-aware message passing in GAT and diffusion models. Node embeddings are shared across all time windows. Graphs are partitioned temporally into training, validation, and test sets to support consistent, time-sensitive model evaluation across datasets with varying density and quality.

3.2 Problem Formulation

We frame link prediction in microservice call graphs as a temporal forecasting task, where the goal is to predict future service-to-service interactions based on historical observations. Specifically, we consider a sequence of time-windowed graphs $\{G_t\}$, where each graph $G_t = (V, E_t)$ represents the set of microservice interactions within the time window t.

Given a graph G_t with learnable node embeddings $x_i \in \mathbb{R}^d$, the model estimates the probability of a future link (i, j) as:

$$p_{ij} = \sigma(x_i^\top x_j),$$

where $\sigma(\cdot)$ is the sigmoid activation function. A decision threshold of 0.5 is applied to classify the presence of a link: if $p_{ij} > 0.5$, the link (i, j) is predicted to exist. This threshold is a standard convention in binary classification tasks with balanced loss and symmetric penalties for false positives and false negatives, and provides a neutral midpoint on the probability scale.

The learning objective is to minimize the binary cross-entropy loss over both positive edges E^+ (observed interactions) and negative edges E^- (non-existent links), sampled using degree-aware negative sampling:

$$\mathcal{L} = -\frac{1}{|E^+|} \sum_{(i,j)\in E^+} \log p_{ij} - \frac{1}{|E^-|} \sum_{(i,j)\in E^-} \log(1 - p_{ij}).$$

In our framework, the model is trained on graphs from earlier time windows and evaluated on graphs from later, unseen windows. This setup reflects a **temporal generalization** setting, where the model forecasts future interactions rather than completing missing links within the same graph.

Notably, we do not use multiple past windows jointly to predict a future one (e.g., $G_{t-n}, \ldots, G_t \rightarrow G_{t+1}$). Instead, each graph window is processed independently, and model generalization across time is achieved through learning over multiple disjoint training windows.

To evaluate the effectiveness of the link prediction, we compute several standard metrics:

- **Area Under the Receiver Operating Characteristic Curve (AUC)** – measures the model's ability to discriminate between positive and negative links;
- **Precision**, **Recall**, and **F1 Score** – assess classification quality;
- **Accuracy** – reports overall prediction correctness;
- **Mean Reciprocal Rank (MRR)** – evaluates the rank of the correct link among predicted candidates.

3.3 Temporal Windowing Strategies

Building on this standardized input, we applied two fundamental strategies to segment the data into temporal windows. Our primary windowing approach used **non-overlapping windows** with a size of 100 ms, where each time slice spans a disjoint interval without temporal overlap.

In addition to this, we implemented a **sliding window strategy with partial overlap**, where each window covers 100 ms of data but advances in 50 ms steps, resulting in a 50% overlap between consecutive windows. This technique generates a denser sequence of snapshots and retains more temporal continuity across windows.

For both approaches, windows are constructed by grouping calls whose timestamps fall within each interval. Each window is then converted into a PyTorch Geometric `Data` object, preserving the edges, edge timestamps (as `edge_attr`), and corresponding node features. These graphs are used for both training and testing depending on their timestamp ranges.

This windowing framework allows flexible experimentation with both granular (e.g., 50 ms) and coarse (e.g., 200 ms, 500 ms) resolutions, as well as overlapping and non-overlapping temporal structures, all while ensuring consistency in graph construction across models.

3.4 Model Architectures

Based on the prepared temporal graphs, we evaluated models in three categories: baseline methods, diffusion-based models, and transformer-based models. Each model operates independently on a sequence of temporal graphs $G_t = (V, E_t)$, where V is the set of microservices and E_t represents the directed interactions within time window t.

Node features $h_i^{(0)}$ are initialized uniformly across graphs using a learnable embedding layer, though alternative strategies such as identity vectors or node degree encodings can be easily substituted. Each model then processes the input graph to compute link probabilities for node pairs, ultimately predicting whether a connection between nodes (i, j) will occur.

Models may optionally use edge attributes during message passing to capture temporal context. Each configuration is evaluated over five random seeds (0-4) to ensure statistical robustness.

Each category of models was selected based on complementary modeling strengths supported by prior work. Baseline models serve as foundational comparisons using classical structural and temporal techniques [15, 16, 33]. Diffusion models simulate smooth feature propagation across dynamic graphs to enhance temporal representations [21, 35]. Transformer-based models are designed to capture long-range temporal dependencies and global interaction patterns across time windows, addressing the limitations of models that rely solely on local neighborhood reasoning [9, 32].

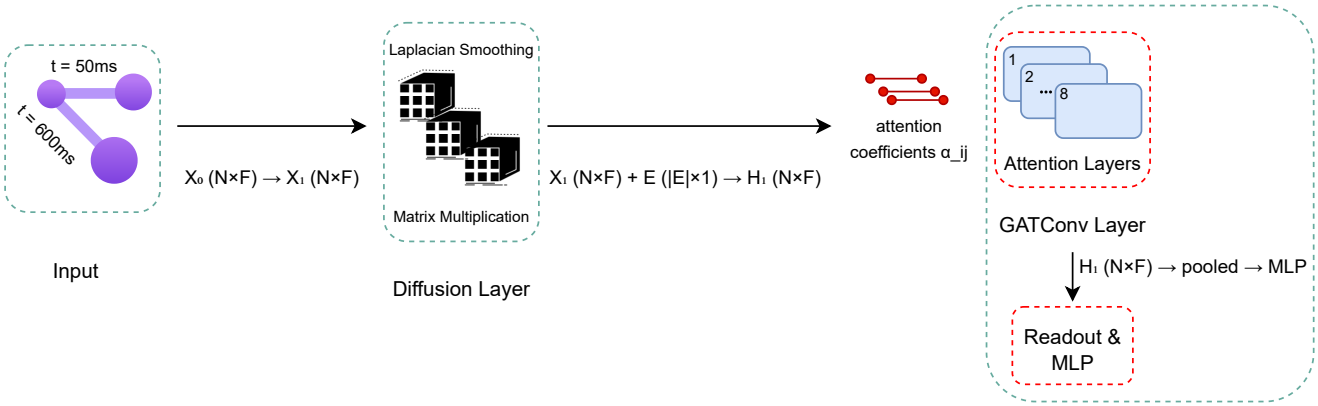

Figure 2: Diffusion Model with GAT: Node features are smoothed, processed by timestamp-aware GAT layers, pooled, and fed into an MLP for link prediction.

3.4.1 Baseline Models Earlier studies have evaluated these baselines; we reuse them for comparability. **NodeSim (Random Walk)** estimates structural proximity through simulated random walks [15]. Although it lacks temporal modeling, it serves as a lightweight, interpretable structure-based baseline for evolving microservice graphs [30]. **LSTM** captures sequential dynamics in microservice call sequences, modeling short- and long-term dependencies effectively [17]. It is used to assess whether standard sequence models are sufficient for time-aware prediction in distributed systems. **GAT (Edge-Aware)** is a Graph Attention Network that employs multi-head attention to learn from neighboring nodes and incorporates timestamps via edge attributes [33], offering a strong baseline that blends structural and partial temporal awareness.

3.4.2 Diffusion Models

Standalone Diffusion The standalone diffusion model is implemented using APPNP [11], which applies iterative feature propagation to simulate local smoothing across graph neighborhoods. Formally, the node representations are updated as:

$$h^{(t+1)} = \alpha \cdot \mathbf{D} \cdot h^{(t)} + (1 - \alpha) \cdot h^{(t)},$$

where $\mathbf{D}$ is the personalized PageRank-based diffusion matrix and α is the teleport (propagation) coefficient. This formulation encourages each node to iteratively aggregate information from its local neighbors while maintaining stability from its initial representation.

Initial node features $h_i^{(0)}$ are sampled from a learnable embedding layer, meaning each microservice is assigned a random vector that is trained end-to-end. These features are passed through a linear layer followed by APPNP propagation.

After propagation, the model computes pairwise link scores between node embeddings using the dot product followed by a sigmoid activation. These scores represent the predicted probability of a link between each node pair in future time windows.

Diffusion + GAT The Diffusion + GAT model enhances node representation learning by first performing Laplacian smoothing on node features through a diffusion layer, followed by refinement using two stacked `GATConv` layers that integrate temporal edge information into their attention mechanisms. As depicted in Figure 2, the diffusion layer initially smooths node embeddings based on adjacency relations. Subsequently, each GAT layer calculates attention coefficients α_{ij} by considering both feature similarity and edge timestamps. Finally, the resulting node embeddings are aggregated through a pooling operation and processed by an MLP classifier.

3.4.3 Transformer-Based Models

Standalone Transformer This model uses stacked transformer encoders operating on positional node embeddings. It models all-to-all interactions across nodes via global self-attention, without using edge attributes directly. This design is effective in capturing long-range dependencies but lacks explicit temporal conditioning from observed interactions.

Transformer + GAT As illustrated in Figure 3, the Transformer + GAT model initially processes node embeddings using Transformer encoders enhanced with positional encodings. These transformed embeddings are subsequently refined through a `GATConv` layer, which leverages edge timestamps to compute attention scores. This hybrid architecture effectively integrates global sequence modeling via self-attention blocks and local, timestamp-aware message passing, enabling the model to adeptly capture both spatial and temporal dynamics.

Among all the models evaluated, only GAT-based architectures (GAT, Diffusion + GAT, and Transformer + GAT) explicitly utilize the `edge_attr` (timestamp) features during message passing, incorporating them into the attention computation.

3.5 Training and Evaluation Protocol

To ensure consistent and fair evaluation, we implement a unified training and testing pipeline configurable at runtime. The framework accepts user-defined parameters including model type, dataset, window size, and training/testing intervals. Based on these, it constructs a temporal graph sequence G_t as described in Section 3.1.

Negative Sampling. We use a degree-aware negative sampling strategy to create challenging false edges. For each positive edge

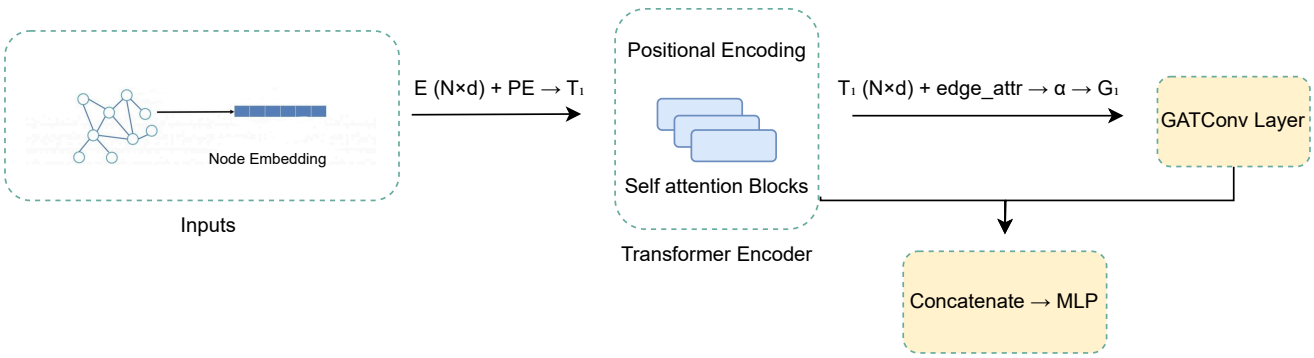

Figure 3: Architecture of the Transformer Model with GAT layer. Node embeddings with positional encodings are processed through transformer encoders. The output is refined via GAT attention incorporating edge timestamps before classification.

$(i, j) \in E_t$, we sample a negative edge $(u, v) \notin E_t$ using a distribution biased toward high-degree nodes (exponent $\alpha = 0.1$). This avoids trivial negatives and improves robustness in dense graphs.

Training. Models are trained using binary cross-entropy loss (Section 3) with the Adam optimizer and weight decay. Early stopping with a patience of 10 epochs is used to prevent overfitting.

Evaluation. We conduct 5-fold cross-validation using disjoint time windows. Evaluation metrics include: **ROC-AUC** (ranking quality), **Precision**, **Recall**, **F1-score** (threshold-based prediction quality), **Accuracy** (overall correctness), and **MRR** (ranking of correct predictions). Graph snapshots and CSV logs are saved for further analysis.

4 Results and Analysis

This section presents the experimental results of our study on link prediction in microservice call graphs. We begin by outlining the experimental setup, followed by a comparison of predictive model performance. The most effective model is then used to analyze key factors such as time window size, train–test gaps, dataset differences, and data scale. These results offer insights for designing efficient temporal models suited to microservice systems.

4.1 Experimental Environment and Reproducibility

All experiments were conducted on Google Colab Pro with Python 3.10 and GPU acceleration (NVIDIA A100, 40 GB VRAM, 83.5 GB RAM). Our implementation is built using PyTorch and PyTorch Geometric, along with standard data processing libraries. The codebase is modular and supports multiple model variants, datasets, and temporal configurations. To support reproducibility and further experimentation, we make our full implementation, processed datasets, and configurations publicly available.[1]

4.2 Model Comparison on Link Prediction

Previous work conducted a comprehensive comparison between a GNN with a GAT core and several baselines, including NodeSim and LSTM, on the task of microservice link prediction. That study demonstrated that the GAT-based model substantially outperformed both structure-driven and temporal baselines [2].

Building on these findings, this evaluation compares the original GAT-based GNN with a diffusion model, a transformer, and their attention-augmented variants (Diffusion+GAT and Transformer+GAT). Each model was trained using 70 time windows and tested using 30 evaluation windows, across five random seeds. All metrics are reported as mean ± standard deviation.

Class Imbalance and Negative Sampling. For each positive edge, one negative edge was sampled per window, resulting in a 1:1 ratio. Threshold-independent (ROC-AUC, MRR) and threshold-dependent (precision, recall, F1, accuracy) metrics are reported.

Table 1 highlights several key findings. **Transformer+GAT** achieved the highest overall performance in terms of F1-score (0.933 ± 0.004), precision (0.889 ± 0.004), and accuracy (0.929 ± 0.004), whereas **Diffusion** led in ROC-AUC (0.980 ± 0.002), recall (0.984 ± 0.003), and MRR (0.920 ± 0.120). **Diffusion+GAT** demonstrated a strong trade-off, attaining high F1 (0.923 ± 0.009) and accuracy (0.919 ± 0.011) while offering faster training than Transformer-based models. In terms of training time per epoch, **GNN (GAT)** was the fastest at 56s, followed by Diffusion+GAT (1m04s), Diffusion (1m12s), Transformer+GAT (1m48s), and Transformer (2m02s). Notably, Transformer+GAT required 104% more training time per epoch than Diffusion+GAT (1m48s vs. 1m04s), despite offering similar predictive performance. Finally, GNN (GAT) delivered competitive accuracy at the lowest computational cost, while Transformer emerged as the most time-intensive model.

Statistical Significance of GAT Attention. Welch's t-tests show statistically significant gains from adding GAT attention:

- Diffusion vs. Diffusion+GAT (Accuracy): $t = -57.22$, $p = 2.21 \times 10^{-125}$
- Diffusion vs. Diffusion+GAT (F1): $t = -61.83$, $p = 4.32 \times 10^{-150}$
- Transformer vs. Transformer+GAT (F1): $t = -66.48$, $p = 2.14 \times 10^{-171}$
- Transformer vs. Transformer+GAT (Accuracy): $t = -72.68$, $p = 6.45 \times 10^{-179}$

[1] https://anonymous.4open.science/r/LinkPrediction-Extention-97B6/README.md

Table 1: Performance of link prediction models (mean ± std over seeds and windows). Best results per metric in bold. Positive-to-negative class ratio: 1:1 via negative sampling.

Model	ROC-AUC	Precision	Recall	F1	Accuracy	MRR	Epoch Training Time
GNN (GAT)	0.972±0.009	0.873±0.008	0.974±0.009	0.921±0.007	0.916±0.008	0.905±0.233	**56s**
Diffusion	**0.980±0.002**	0.672±0.033	**0.984±0.003**	0.799±0.022	0.751±0.035	**0.920±0.120**	1m12s
Diffusion+GAT	0.970±0.005	0.877±0.016	0.975±0.005	0.923±0.009	0.919±0.011	0.900±0.200	1m04s
Transformer	0.958±0.012	0.777±0.010	0.951±0.017	0.855±0.012	0.839±0.013	0.900±0.150	2m02s
Transf.+GAT	0.974±0.007	**0.889±0.004**	0.980±0.003	**0.933±0.004**	**0.929**±0.004	0.820±0.360	1m48s

4.3 Effect of Temporal Window Size

Following the model comparison, **Diffusion+GAT** was selected for further evaluation due to its strong balance of predictive accuracy and computational efficiency. All results in this section are reported as the mean over five random seeds, with models trained on the first 7000 ms and evaluated on the following 3000 ms.

To assess the effect of temporal granularity, we compared five windowing strategies: four non-overlapping window sizes (50 ms, 100 ms, 200 ms, 500 ms) and one overlapping configuration (100 ms with 50% overlap).

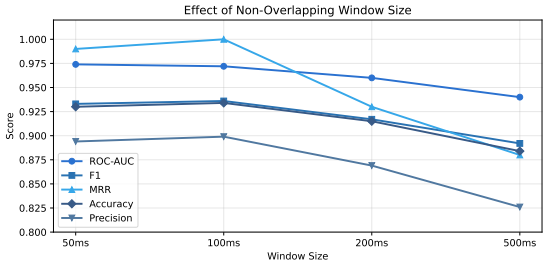

Figure 4: Effect of non-overlapping temporal window size on key metrics (Diffusion+GAT).

As shown in Figure 4, performance improves from 50 ms to 100 ms windows, followed by a consistent decline at 200 ms and 500 ms. This trend holds across all major metrics including ROC-AUC, F1, precision, and accuracy. Introducing 50% overlap in the 100 ms configuration yields modest but consistent gains(Figure 5).

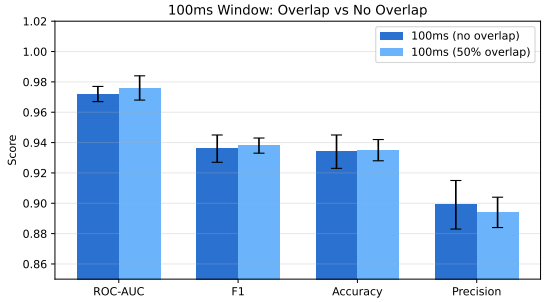

Figure 5: Comparison of 100 ms windows with and without 50% overlap.

Standard deviations across five seeds were consistently low. For example, the ROC-AUC standard deviation was 0.005 for 50 ms, 0.002 for 100 ms, 0.007 for 200 ms, and 0.009 for 500 ms. F1-score varied by less than 0.011, accuracy remained within a 0.016 range, and MRR and precision also showed limited fluctuation, confirming robustness and reproducibility.

Overall, the results show that temporal window size strongly impacts performance. Smaller windows (up to 100 ms) improve precision and granularity, while larger ones blur temporal signals. Overlapping windows capture boundary-spanning interactions, offering slight gains over non-overlapping setups. These effects are discussed further in the Discussion section.

4.4 Effect of Training-Testing Gap

To evaluate the temporal generalization capability of our best-performing model (Diffusion+GAT), we conducted experiments with increasing time gaps between training and testing. The model was trained on data from 0 ms to 7000 ms and evaluated on the following test intervals: 7000–10000 ms (no gap), 8000–11000 ms (1-second gap), 10000–13000 ms (3-second gap), 175000–178000 ms (approximately 3-minute gap), 540000–543000 ms (approximately 10-minute gap), and 180000–183000 ms (on the third day).

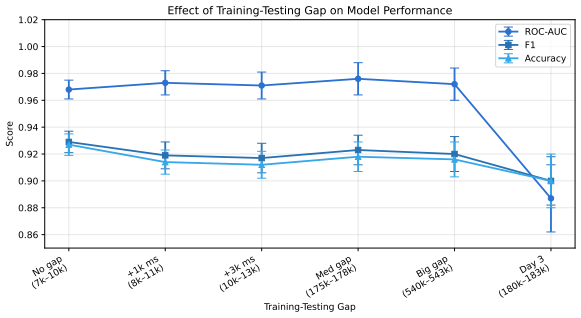

Figure 6: Impact of increasing training-testing gap on key metrics (Diffusion+GAT).

For each gap configuration, we report average performance across windows within the test interval and five random seeds. A clear pattern emerges: performance remains stable with minimal or no gap, and gradually declines as the temporal distance increases.

In the no-gap setting (7000–10000 ms), metrics are consistently high, with F1 and ROC-AUC both exceeding 0.92 and 0.96, respectively. With a 1-second and 3-second gap, the drop is small. However, when the gap exceeds multiple minutes, the decline becomes more evident: AUC can fall below 0.85 and F1 below 0.90, particularly in the test range on the third day. The decline is also accompanied by increased variability across test windows.

Recall remains comparatively higher than other metrics across all temporal distances, while precision, F1, accuracy, and AUC show more sensitivity to time gaps.

These results confirm that Diffusion+GAT performs best when trained and tested in close temporal proximity. The detailed reasoning and implications behind this behavior are discussed further in the next section.

4.5 Temporal Fluctuations and Recurring Link Activity

To investigate recurring patterns in microservice interactions, we analyzed link density over time in the Alibaba 2022 dataset. We computed the number of service-to-service interactions in each 3-second window from 425s to 600s.

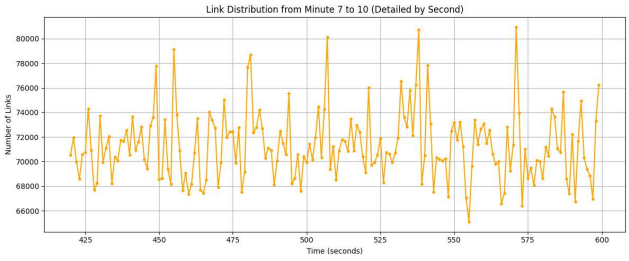

Figure 7: Link distribution: Peaks at 571s, dips at 555s.

As illustrated in Figure 7, active links fluctuate across the interval, with distinct peaks and troughs that suggest recurring load patterns. Based on this distribution, we selected two representative windows: a low-density period at 555s with reduced service call volume, and a high-density window at 571s featuring increased activity and structural complexity.

To assess generalization under shifting load conditions, we trained Diffusion+GAT on the low-density interval (555s) and evaluated it on the high-density interval (571s). Despite the change in workload, the model maintained strong performance, achieving AUC ∼0.95 and F1 > 0.90. This indicates that the model can transfer learned features from lighter to heavier load conditions while preserving predictive accuracy, even across structurally distinct but recurring behavioral patterns.

These results align with observations from *An In-Depth Study of Microservice Call Graph and Runtime Performance* [25], which documents similar cyclic and event-driven behaviors in production environments. That study provides supporting evidence for our generalization setup by analyzing the frequency of recurring patterns, temporal locality, and service-level scheduling dynamics.

4.6 Dataset Comparison

To evaluate the robustness and generalizability of the proposed approach, experiments were conducted using three real-world microservice call graph datasets: Alibaba 2022, Alibaba 2021, and Huawei 2021.

Alibaba 2022 [14] covers 8 hours of production traffic from Alibaba's large-scale cloud infrastructure. It reflects a containerized microservice environment with thousands of services on Kubernetes, including logs of inter-service calls, resource usage, and container lifecycle events.

Alibaba 2021 [13] is an earlier 6-hour snapshot of the same platform, with fewer services and more stable traffic. It contains traces of service dependencies, scheduling decisions, and resource metrics.

Huawei 2021 [5] is a 24-hour trace from Huawei Cloud's internal microservice platform. It includes logs of inter-service calls and system metrics. Compared to the Alibaba traces, it has more overlapping timestamps, higher noise, and a more heterogeneous service topology.

Results. Mean scores for ROC-AUC, F1, precision, recall, accuracy, and MRR are averaged over 30 test windows. For **Alibaba 2022**, Diffusion+GAT achieves consistently high performance with both AUC and F1 close to 0.97, and accuracy exceeding 0.92. In **Alibaba 2021**, performance slightly improves in some windows with F1 reaching up to 0.96, though occasional dips in AUC and accuracy are observed. For the **Huawei 2021** dataset, overall performance declines, with AUC around 0.57, F1 near 0.68, and accuracy around 0.54, although recall remains high (> 0.99) across most test windows.

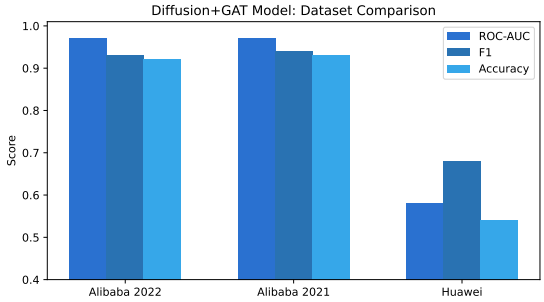

Figure 8: Comparison of Diffusion+GAT model performance across three datasets.

Figure 8 visualizes these results, clearly showing differences in average AUC, F1, and accuracy across datasets. The Alibaba datasets produce similarly high scores, while Huawei 2021 yields significantly lower performance. Further interpretation of these differences is discussed in the next section.

4.7 Inference Efficiency and Metadata Collection

To assess the feasibility of real-time prediction, we recorded detailed metadata for each experimental run, including training and inference durations, early stopping epochs, and the number of test windows evaluated. For every seed and temporal fold, the following information was stored: the Seed and Fold Index, which identifies the training configuration; EarlyStopEpoch, indicating the epoch when early stopping was triggered based on validation loss; TrainTimeSec and EvalTimeSec, which represent the total durations of training and evaluation in seconds; and NumTestWindows, denoting the number of testing windows used for link prediction.

Inference Durations. We measured end-to-end evaluation time for predicting 3 seconds of future interactions from 7 seconds of history. Average runtimes across seeds and folds were: GNN (GAT) ~43 min, Diffusion+GAT ~1h07m, standalone Diffusion ~1h25m, Transformer+GAT ~1h46m, and standalone Transformer ~2h33m. These include data loading, graph construction, and inference. Although the A100 GPU offered faster performance than L4, neither setup achieved real-time latency under the current implementation.

5 Discussion

Each subsection presents a research question, followed by its results and rationale.

5.1 RQ1: Model Performance for Link Prediction

Question 1: Which model performs best for link prediction in microservice call graphs?
Answer: The Transformer+GAT and Diffusion+GAT models perform best due to their integration of attention mechanisms, which significantly improve their effectiveness in capturing temporal and structural relationships. Specifically, Diffusion+GAT is optimal for microservice call graphs given its balance between precision and computational efficiency.

To contextualize model performance, Table 2 summarizes architectural properties across temporal awareness, structural reasoning, edge feature usage, scalability, and design motivations, with additional detail in Section 2.

Baseline models such as NodeSim and LSTM demonstrate clear limitations. NodeSim focuses only on structural proximity and lacks temporal context, while LSTM captures sequential patterns but cannot reason over graph structure. These constraints hinder their ability to model complex, evolving microservice interactions.

GAT-based models outperform baselines by applying attention to highlight relevant neighbors and suppress noisy edges, improving prediction quality. GNNs aggregate local information, diffusion models propagate signals smoothly across the graph, and transformers model long-range dependencies. However, transformers exhibit quadratic complexity, limiting their scalability in high-throughput environments.

Diffusion+GAT combines the strengths of both diffusion and attention. It captures gradual topological evolution while attending to critical short-term dynamics, aided by its use of edge-level timestamps as features. This architectural synergy proves particularly effective on Alibaba 2022, where frequent bursts and dynamic changes demand both stability and sensitivity.

Notably, GAT-augmented models consistently achieve high recall and MRR, meaning they rank true links near the top even when predictions are imperfect. This robustness is valuable in operational scenarios where top-k predictions trigger preemptive actions.

While Transformer-based models perform well in settings requiring long-term temporal modeling, their computational overhead outweighed their benefits in our real-time evaluations. By contrast, Diffusion+GAT offered a favorable trade-off between accuracy and efficiency, generalizing well across time windows and deployment conditions. Its consistent performance on structurally stable yet temporally volatile datasets supports its practical utility for predictive monitoring in real-world microservice systems.

5.2 RQ2: Impact of Time Windowing Strategies

Question 2: How do different time windowing strategies impact model performance on a dynamic call graph?

Answer: Different time windowing strategies impact model performance significantly by affecting the granularity of temporal information captured. In our case, a 100 ms window with partial overlap achieves an optimal balance between temporal resolution and computational stability.

An important factor influencing model effectiveness is the time windowing strategy used during training and inference. In microservice systems, where traffic patterns shift rapidly due to autoscaling and short-lived service dependencies, selecting the right window size is critical for capturing temporal dynamics.

Our results show that a 100 ms non-overlapping window strikes a good balance: it preserves short-term dependencies without excessive fragmentation, supporting both context and computational efficiency. Partial overlap further improves performance (Figure 5) by restoring continuity across window boundaries. This shared context enhances models like Diffusion+GAT, which benefit from tracking evolving patterns across segments.

In contrast, larger windows tend to oversmooth transient signals, weakening the predictive power of short-range dependencies. Strictly non-overlapping windows risk breaking continuity, especially when key interactions fall near window edges.

Thus, combining 100 ms windows with partial overlap captures high-fidelity temporal signals while aligning with the model's strengths in edge-aware and structure-aware reasoning. This setup enables more accurate and timely link predictions in dynamic, cloud-native environments.

5.3 RQ3: Temporal Patterns and Recurring Connections

Question 3: Are there recurring patterns in microservice connections over time?
Answer: Yes, microservice interactions exhibit temporal patterns that can be leveraged to improve prediction. These patterns often recur across time windows, providing opportunities for models to learn transferable structural and temporal behaviors.

Temporal regularity is a hallmark of many microservice systems. Prior work [25] shows that service behaviors often follow cyclical or event-driven patterns. These patterns often emerge from recurring workload surges, batch processing jobs, autoscaling routines, or scheduled operational tasks. As a result, certain interaction subgraphs tend to reappear over time, even if the system experiences different load conditions.

This recurrence of structural motifs enables learning-based models to generalize across temporally distant intervals. Once a model

Model	Temporal	Structure	Edge Features	Scalability	Motivation
NodeSim (RW)	None	Proximity	No	High	Random walk baseline [15]
LSTM	Sequential	None	No	High	Sequential patterns [18]
GAT (Edge-Aware)	Windowed	Attention	Partial	Moderate	Neighborhood importance [33]
Simple Diffusion	Local	Propagation	No	High	Local diffusion [21]
Diffusion + GAT	Smoothed	Weighted	Yes	Moderate	Propagation + attention [28]
Simple Transformer	Global	Full	No	Low	Long-range dependencies [32]
Transformer + GAT	Global	Edge-aware	Yes	Low	Global + edge context [9]

Table 2: Model characteristics comparison.

captures these repeatable patterns, it can apply them in future windows—even when those windows differ in service activity or intensity. This property is particularly important in dynamic cloud environments, where load fluctuations are frequent but underlying architectural relationships remain stable.

The ability of our model to maintain high predictive performance when trained on low-density periods and evaluated on high-density ones reflects this pattern stability. The GAT component attends to the most relevant connections, emphasizing persistent structural dependencies rather than raw interaction frequency. Meanwhile, the diffusion component facilitates representation smoothing across local neighborhoods, making the model robust to bursts or sparsity. Together, these architectural features help the model generalize across different operating regimes without overfitting to specific traffic volumes.

5.4 RQ4: Temporal Generalization

Question 4:

(1) How does training on one time period and testing on another affect model generalization?
(2) How do temporal gaps between training and testing affect link prediction performance?

Answer: Performance declines when training and testing occur in different time periods due to evolving interaction patterns. Although wider temporal gaps reduce accuracy, the model maintains reasonable performance even under significant shifts.

Temporal generalization poses a key challenge in dynamic graph prediction, especially in microservice systems where interaction patterns evolve rapidly due to load shifts, service updates, and deployment cycles. Our results show that performance degrades gradually as the time gap between training and testing widens. This reflects distributional drift in service interactions, where structural dependencies shift over time despite stable high-level topologies.

As noted in Section 4.5, models trained during low-density periods still generalized well to high-density intervals, indicating robustness across varying load conditions. However, larger time gaps—such as training on early traces and testing much later—led to more pronounced performance drops. This suggests that short-term dependencies are crucial, and that models are sensitive to evolving interaction frequencies and graph structures.

To sustain high accuracy over longer horizons, periodic retraining or online adaptation may be necessary to align with the system's changing behavior.

5.5 RQ5: Cross-Dataset Performance

Question 5: How do different datasets from various systems influence model performance?
Answer: Model performance varies significantly across datasets due to inherent structural and temporal differences. Alibaba 2022 yields the highest performance, while Huawei 2021 exhibits substantial performance degradation, highlighting dataset-specific challenges.

Analyzing model performance across the three datasets reveals how system behavior, trace quality, and structural regularity influence prediction accuracy.

Alibaba 2022 yields the strongest results, likely due to its clean traces, stable microservice interactions, and regular call patterns. These characteristics enable models to effectively learn and generalize temporal and structural features, resulting in high AUC, F1, precision, and MRR.

Alibaba 2021, though similar in origin, exhibits greater performance variability, potentially due to intermittent drops in call density or irregular service behaviors. Nevertheless, it maintains strong generalization, indicating a generally well-structured environment.

In contrast, **Huawei 2021** presents significant challenges. Its noisy and potentially sparse traces result in lower precision and accuracy. Although recall remains high, indicating that the model captures most potential links, the substantial number of false positives indicates reduced discriminative power. This reduced performance may arise from class imbalance, inconsistent interaction patterns, or weak graph structure.

Additionally, Huawei's overlapping interactions with nearly identical timestamps complicate the ability of attention mechanisms to prioritize meaningful edges, thereby diminishing their effectiveness in capturing relevant signals.

Overall, the results highlight the importance of dataset quality and temporal structure. While Diffusion+GAT generalizes well in clean, stable environments like Alibaba, its performance degrades in noisy or irregular systems, pointing to the need for dataset-specific preprocessing or adaptive modeling strategies.

5.6 RQ6: Real-time Link Prediction Feasibility

Question 6: Can we predict links in real-time, and which model is best suited for live prediction?
Answer: While some models like Diffusion+GAT offer strong predictive accuracy, true real-time prediction is not feasible under our current setup due to high inference latency. Among the evaluated models, the standard GNN is the most time-efficient and may be better suited for live prediction in latency-sensitive systems.

The feasibility of real-time link prediction hinges not only on predictive accuracy, but also on end-to-end system latency. Despite the strong performance of models such as *Diffusion+GAT* in terms of generalization and temporal robustness, their computational demands render true real-time inference impractical under our current pipeline.

The bottleneck stems from the sequential nature of temporal window processing and the time-consuming operations involved in graph construction and feature aggregation. Even with modern GPU acceleration, the inference delay exceeds the time horizon being predicted, making live deployment infeasible.

Among all architectures, the standard GNN (GAT-only) showed the greatest runtime efficiency. While it sacrifices some predictive accuracy, its significantly lower inference time makes it a viable option for scenarios where latency constraints outweigh marginal performance gains.

These findings indicate that our current framework is better suited for near-real-time or batch-based forecasting systems, where modest delays (e.g., tens of minutes) are acceptable. For use in strict real-time applications, future work should focus on hardware optimization or algorithmic acceleration techniques such as streaming inference, model pruning, or quantization to reduce the end-to-end processing time.

Overall, our findings highlight the trade-offs between model complexity, temporal resolution, and real-world applicability. Diffusion+GAT consistently delivers strong predictive performance, but its computational cost limits real-time deployment. In contrast, lighter models sacrifice accuracy for speed, suggesting a need for adaptive solutions in latency-sensitive environments.

6 Threats to Validity

Internal Validity. Our negative sampling assumes that unobserved links are truly negative. However, some may represent rare or delayed interactions, introducing label noise. While we used fixed hyperparameters and validated across five random seeds, models may still exhibit sensitivity to initialization or specific training-test splits. Hidden leakage (e.g., service name reuse) could also influence results if not properly filtered.

External Validity. While our datasets reflect real-world production systems, they may not generalize to other microservice environments such as edge computing or federated systems. Notably, Diffusion+GAT struggles on sparse or noisy traces like Huawei 2021, where AUC drops to 0.57 and accuracy to 0.54. These differences underscore the need for broader validation across heterogeneous infrastructures.

Construct Validity. We report standard metrics such as ROC-AUC, F1, and MRR under a 1:1 class balance. While this supports comparability, real-world deployments are often highly imbalanced and sensitive to false positives. Our metrics may not fully capture practical factors like alert fatigue, latency tolerance, or prioritization needs.

Infrastructure Validity. Inference time results were obtained using an A100 GPU on Google Colab Pro+. For example, generating predictions for a 3-second interval takes over 1 hour—well above the 100 ms window rate. This challenges real-time deployment, especially in resource-constrained or batch-inference settings.

By acknowledging these limitations, we aim to encourage careful interpretation of our findings and identify directions for strengthening generalizability and deployment-readiness.

7 Conclusions and Future Work

This paper presents a comprehensive evaluation of link prediction models for dynamic microservice call graphs, focusing on architectural design, temporal generalization, scalability, and real-time feasibility. Across extensive experiments, we find that hybrid models—especially Diffusion+GAT—offer an effective trade-off between predictive accuracy and efficiency.

Diffusion+GAT achieved an F1-score of 0.923, improving over vanilla GAT by +1.2 percentage points while reducing per-epoch training time by 38%. It generalizes well across datasets with consistent structure, remains robust under varying time gaps, and performs competitively even in coarse or overlapping window regimes.

At the same time, challenges remain for real-time inference and generalization in noisy or sparse settings. While Diffusion+GAT offers significant progress, further optimization is needed to support live deployment in production systems.

To this end, future work will explore hardware-aware techniques such as quantization, pruning, and streaming inference to reduce latency. We also plan to investigate adaptive learning strategies (e.g., transfer learning, continual learning) for improved performance on heterogeneous environments. Lightweight, deployment-friendly architectures will also be considered for latency-sensitive applications.

References

[1] Lada A Adamic and Eytan Adar. 2003. Friends and neighbors on the web. *Social networks* 25, 3 (2003), 211–230.
[2] Anonymous. 2025. Citation anonymized for double-blind review..
[3] Tianchi Chen, Yifan Wu, Kai Li, and Ping Tan. 2018. GC-LSTM: Graph Convolution Embedded LSTM for Dynamic Link Prediction. *arXiv preprint arXiv:1806.02906* (2018).
[4] Xin Chen et al. 2021. A comprehensive survey on tracing and monitoring microservices. *Comput. Surveys* 54, 4 (2021), 1–36.
[5] Huawei Cloud. 2021. Huawei Cloud microservice trace dataset. https://zenodo.org/record/5638238. Accessed: July 2025.
[6] Nicola Dragoni, Saverio Giallorenzo, Alberto Lluch Lafuente, Manuel Mazzara, Fabrizio Montesi, Ruslan Mustafin, and Larisa Safina. 2017. Microservices: Yesterday, today, and tomorrow. *Present and Ulterior Software Engineering* (2017), 195–216.
[7] Xudong Du et al. 2021. Graph-based modeling and analysis of microservices architectures. *IEEE Transactions on Services Computing* (2021).
[8] Vijay Prakash Dwivedi, Sri Jaladi, Yangyi Shen, Federico López, Charilaos I Kanatsoulis, Rishi Puri, Matthias Fey, and Jure Leskovec. 2025. Relational Graph Transformer. *arXiv preprint arXiv:2505.10960* (2025).
[9] Vijay Prakash Dwivedi, Chaitanya K Joshi, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. 2020. A Generalization of Transformer Networks to Graphs. *arXiv preprint arXiv:2012.09699* (2020).

[10] Martin Fowler and James Lewis. 2014. Microservices: a definition of this new architectural term. *martinfowler.com* (2014).

[11] Johannes Gasteiger, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Predict then propagate: Graph neural networks meet personalized pagerank. *arXiv preprint arXiv:1810.05997* (2018).

[12] Anna Golovkina, Dmitry Mogilnikov, and Vladimir Ruzhnikov. 2024. Graph Neural Networks for Metrics Prediction in Microservice Architecture. In *International Conference on Computational Science and Its Applications*. Springer, 343–357.

[13] Alibaba Group. 2021. Alibaba Cluster Data: Microservices Trace 2021. https://github.com/alibaba/clusterdata/tree/master/cluster-trace-microservices-v2021. Accessed: July 2025.

[14] Alibaba Group. 2022. Alibaba Cluster Data: Microservices Trace 2022. https://github.com/alibaba/clusterdata/tree/master/cluster-trace-microservices-v2022. Accessed: July 2025.

[15] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable Feature Learning for Networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 855–864.

[16] William L Hamilton, Rex Ying, and Jure Leskovec. 2017. Representation Learning on Graphs: Methods and Applications. *IEEE Data Engineering Bulletin* 40, 3 (2017), 52–74.

[17] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation* 9, 8 (1997), 1735–1780.

[18] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.

[19] Alibaba Inc. 2021. clusterdata - cluster-trace-microservices-v2021. https://github.com/alibaba/clusterdata/tree/master/cluster-trace-microservices-v2021. Accessed: 2025-05-28.

[20] Alibaba Inc. 2022. clusterdata - cluster-trace-microservices-v2022. https://github.com/alibaba/clusterdata/tree/master/cluster-trace-microservices-v2022. Accessed: 2025-05-28.

[21] Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. 2019. Diffusion Improves Graph Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*. 13366–13378.

[22] Yang Li et al. 2020. DeepTrace: Enabling link prediction in microservice tracing graphs. In *IEEE International Conference on Cloud Computing*. 170–178.

[23] David Liben-Nowell and Jon Kleinberg. 2007. The link-prediction problem for social networks. *Journal of the American Society for Information Science and Technology* 58, 7 (2007), 1019–1031.

[24] Linyuan Lü and Tao Zhou. 2011. Link prediction in complex networks: A survey. *Physica A: statistical mechanics and its applications* 390, 6 (2011), 1150–1170.

[25] Shutian Luo, Huanle Xu, Chengzhi Lu, Kejiang Ye, Guoyao Xu, Liping Zhang, Jian He, and Chengzhong Xu. 2022. An in-depth study of microservice call graph and runtime performance. *IEEE Transactions on Parallel and Distributed Systems* 33, 12 (2022), 3901–3914.

[26] Víctor Martínez, Fernando Berzal, and Juan Carlos Cubero. 2016. A survey of link prediction in complex networks. *ACM Computing Surveys (CSUR)* 49, 4 (2016), 1–33.

[27] Sam Newman. 2015. *Building Microservices*. O'Reilly Media, Inc.

[28] Emanuele Rossi and et al. 2020. Temporal graph networks for deep learning on dynamic graphs. *arXiv preprint arXiv:2006.10637* (2020).

[29] Gianluca Ruberto. 2022. An experimental analysis of Link Prediction methods over Microservices Knowledge Graphs. (2022).

[30] Akrati Saxena, George Fletcher, and Mykola Pechenizkiy. 2022. NodeSim: node similarity based network embedding for diverse link prediction. *EPJ Data Science* 11, 1 (2022), 24.

[31] Da Sun Handason Tam, Yang Liu, Huanle Xu, Siyue Xie, and Wing Cheong Lau. 2023. Pert-gnn: Latency prediction for microservice-based cloud-native applications via graph neural networks. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2155–2165.

[32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*. 5998–6008.

[33] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations (ICLR)*.

[34] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations (ICLR)*.

[35] Bingbing Xu, Huawei Shen, Qi Cao, Keting Cen, and Xueqi Cheng. 2020. Graph-Heat: Diffusion-Based Graph Neural Networks for Node Classification. *IEEE Transactions on Neural Networks and Learning Systems* 32, 10 (2020), 4812–4823.

[36] T. Yang. 2021. AID Dataset. https://zenodo.org/record/5638238. https://doi.org/10.5281/zenodo.5638238 Accessed: 2025-05-28.

[37] Rex Ying, Ruining He, Kaifeng Chen, Piriya Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 974–983.

[38] Chuan Zhang, Lei Zhang, and Philip S Yu. 2018. Link prediction across multiple social networks. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 12, 4 (2018), 1–31.

[39] Xin Zhang et al. 2019. Deep learning-based anomaly detection for call graphs in microservices. In *IEEE/ACM International Conference on Software Engineering (ICSE)*. 1–12.

[40] Yu Zhou et al. 2020. Learning service dependencies from microservice traces using GNNs. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 760–771.